

프로그래밍 역량 강화 전문기관, 민코딩

클린코드를 위한 OOP 개요



목차

1. 객체지향 패러다임의 이해
2. 객체지향 vs 절차지향
3. 의존도
4. Attribute 와 Behavior
5. OOP개요1
6. OOP개요2
7. 클래스 다루기
8. 캡슐화
9. 상속
10. Interface 개요
11. Interface 구현
12. 다형성 구현
13. 확장성 고려한 Class 제작 Mission

Chapter1

객체지향 패러다임의 이해

클린코드의 정의

1. 가독성 좋은 코드
2. 안전한 코드
3. 유지보수성이 좋은 코드

유지보수성이 좋은 코드는

객체지향 패러다임을 적용한 코드이다.

- 패러다임 : 생각하는 방식
- 객체를 중심으로 생각하는 방식을 "**객체지향 패러다임** 이라고 한다"

객체지향 패러다임 설계 예시 1

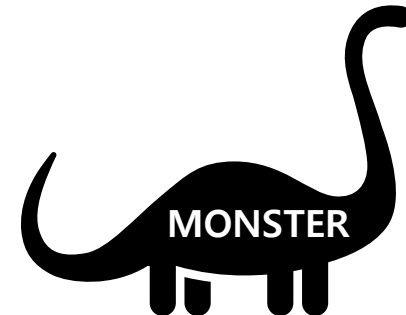
객체지향 패러다임

객체를 중심으로 생각하는 방식

객체지향 패러다임 1 단계 : 구성요소 떠올리기

간단한 게임을 만들고자 한다.

내가 만들고자 하는 프로그램을 구성하는 요소들을 먼저 떠올려본다.



객체지향 패러다임 설계 예시 2

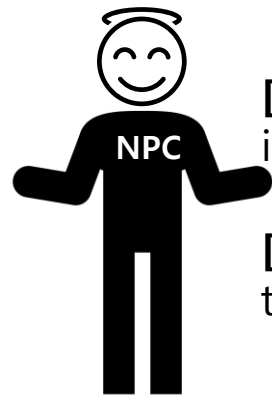
객체지향 패러다임 2 단계 : 데이터와 동작 결정하기

각 모듈들이 어떤 데이터를 가지고 있고,
어떤 동작을 할 수 있는지 결정한다.



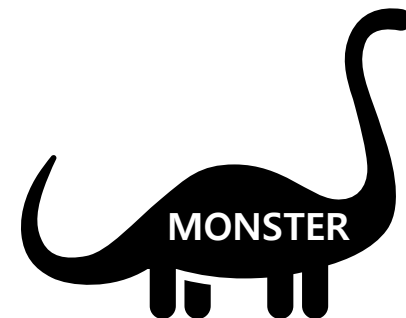
[데이터]
HP, MANA

[동작]
attack()
move()
talk()



[데이터]
isTalk

[동작]
talk()



[데이터]
HP

[동작]
attack()
die()

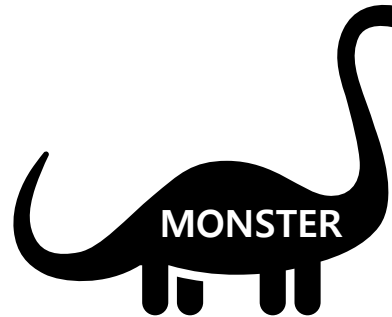
객체지향 구현 예시 1



[데이터]
HP, MANA

[동작]
attack()
move()
talk()

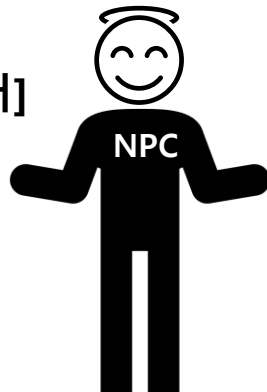
```
class User {  
    //state (Java에서는 필드)  
    int HP;  
    int mana;  
  
    //behavior (Java에서는 메서드)  
    void attack() { ... }  
    void move() { ... }  
    void talk() { ... }  
}
```



[데이터]
HP

[동작]
attack()
die()

```
class Monster {  
    //state (Java에서는 필드)  
    int HP;  
  
    //behavior (Java에서는 메서드)  
    void attack() { ... }  
    void die() { ... }  
}
```



[데이터]
isTalk

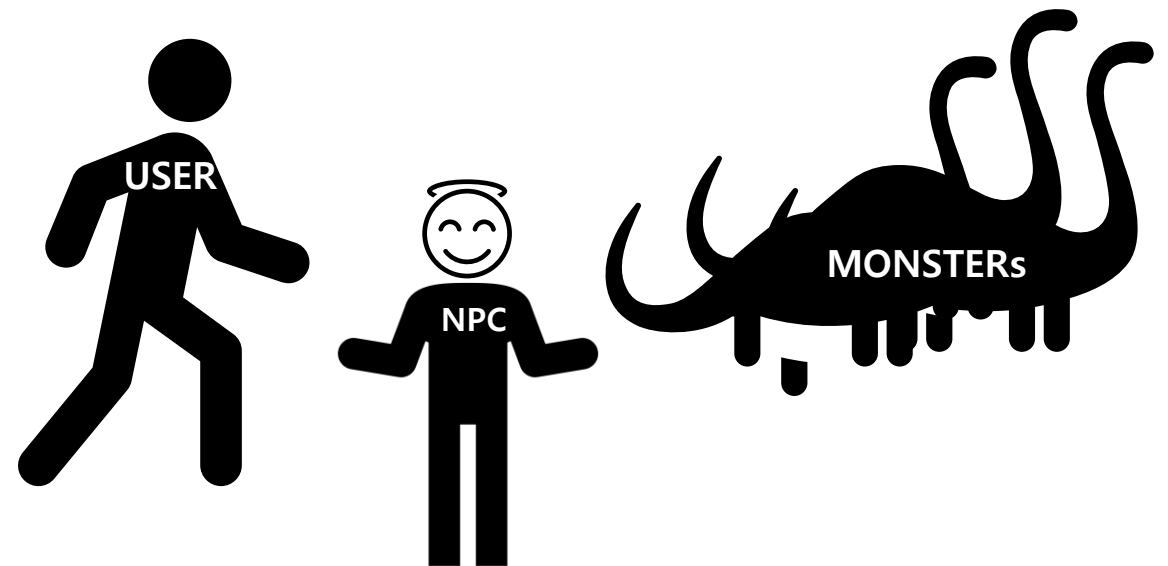
[동작]
talk()

```
class Npc {  
    //state (Java에서는 필드)  
    int isTalk;  
  
    //behavior (Java에서는 메서드)  
    void talk() { ... }  
}
```

객체지향 구현 예시 2

객체지향 패러다임으로 설계된 객체들을
생성 & 제어하는 코드를 추가하여 프로그램을 동작시킨다.

```
class Main {  
    public static void main(String[] args) {  
        User user1 = new User();  
        Npc npc = new Npc();  
  
        ArrayList<Monster> monsters = new ArrayList<>();  
  
        user1.move();  
        user1.talk();  
        npc.talk();  
  
        user1.move();  
        user1.attack();  
  
        for (var mon : monsters) {  
            mon.attack();  
        }  
    }  
}
```



Chapter2

절차지향 vs 객체지향

무한 Loop 속 순서를 정하여 구현한다.

머릿속으로 전체를 시뮬레이션 해보면서
어디에 넣어야 정상 동작하는지
고민하면서 코딩한다.

```
while(1)
{
    //1. 버튼인식

    //2. 만약 stop 일 때
    모터stop( )

    //3. 만약 1 단계
    모터ON( )

    //4. 만약 2 단계
    모터 ON( )
    전력 UP( )
}
```

“손가락이 감지되면 즉시 멈춘다 “
코드를 어디에 넣어야할까?

개발자마다
유지보수 위치가 다를 가능성이 크다.

```
while(1)
{
    //1. 버튼인식
    //2. 만약 stop 일 때
    모터stop( )
    //3. 만약 1 단계
    모터ON( )
    //4. 만약 2 단계
    모터 ON( )
    전력 UP( )
}
```



객체지향 : 실제와 비슷하게 설계

컨트롤러

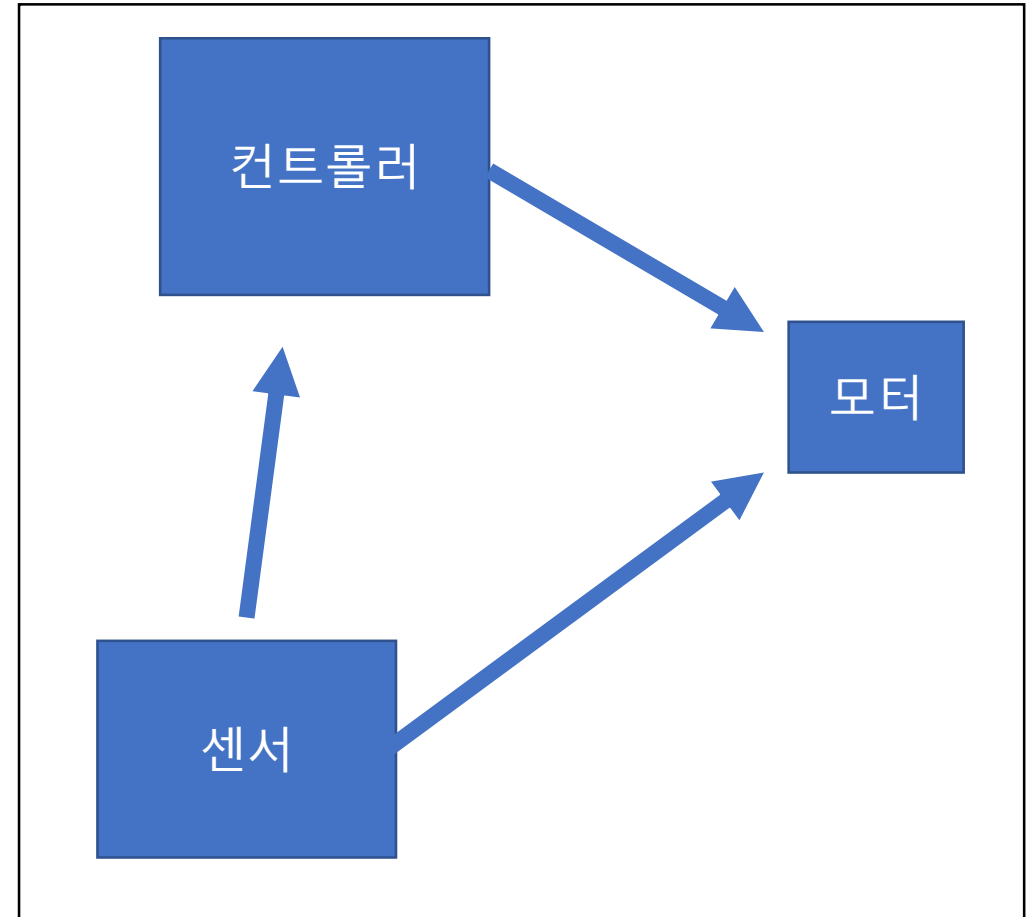
버튼이 눌리면 모터에게 메세지 보낸다.

모터

누군가에게 메세지 받으면 동작한다.

센서

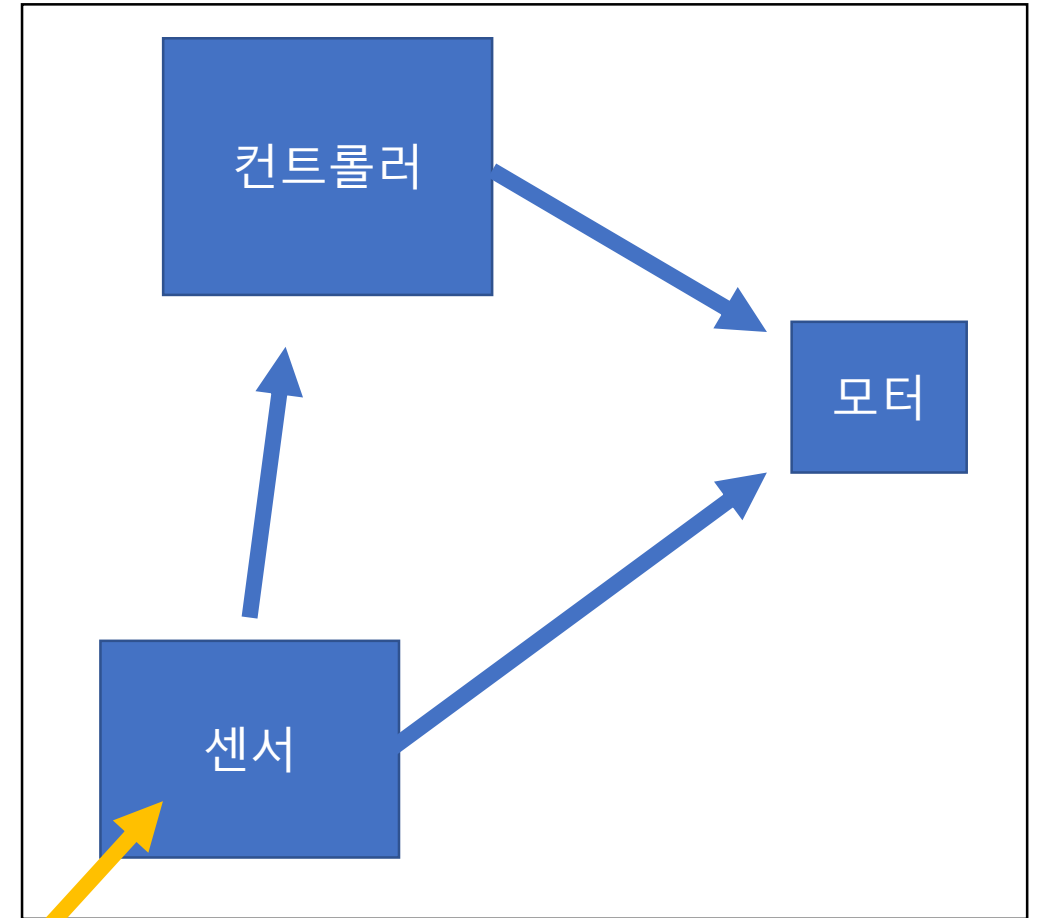
모터가 과부하되면 컨트롤러에게
Stop메세지를 보낸다.



객체지향 : 유지보수성이 좋다.

모듈화가 잘 되어있다면,
유지보수성이 좋아진다.

1. 버그가 발생했을 때,
더 빠르게 디버깅이 가능하다.
2. 기능을 추가할 때,
유지보수 위치가 예측 가능해진다.



“손가락이 감지되면 즉시 멈춘다 “
기능을 이곳에 추가한다.

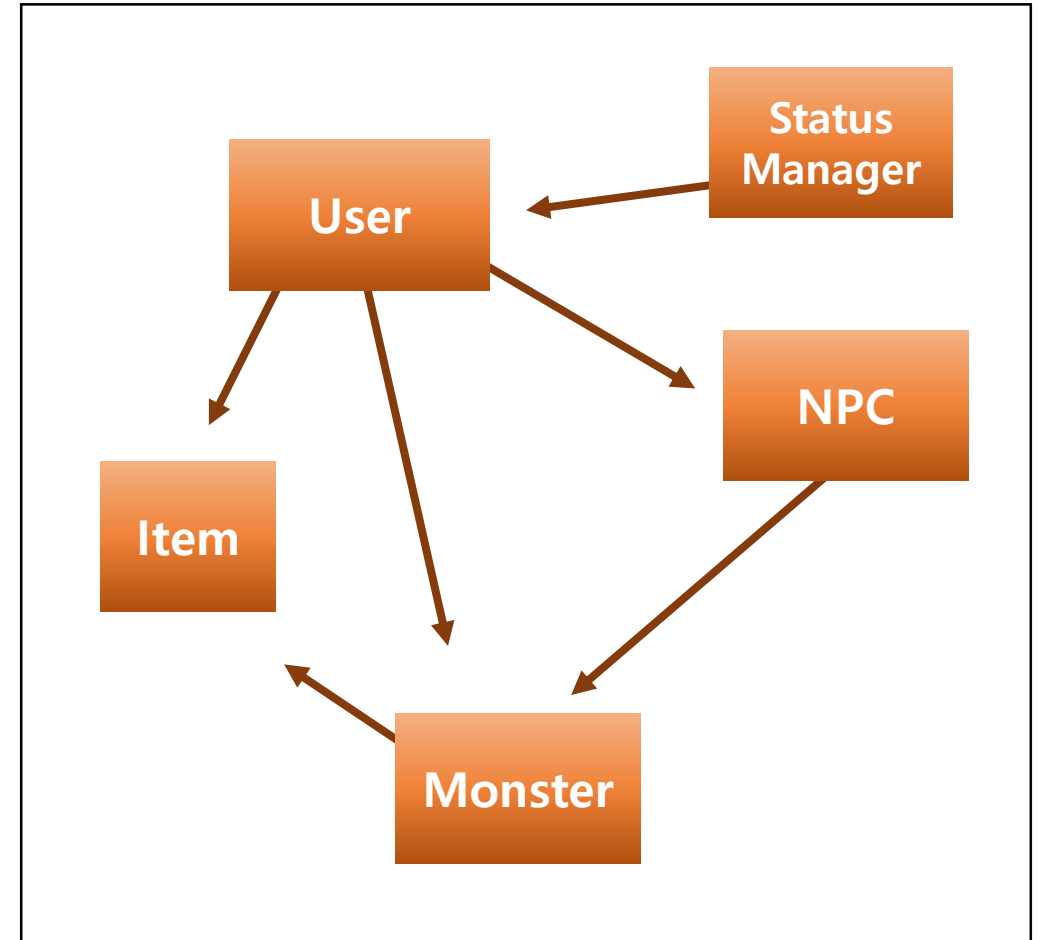
객체지향의 정의

하나의 시스템은 여러 모듈들로 구성되어있고
모듈들이 메시지를 주고 받으면서
전체 모듈이 하나의 시스템을 구성한다.

각 모듈은 본인의 역할에만 충실한다.



엘런케이(Alan Kay)
객체지향 패러다임의 아버지

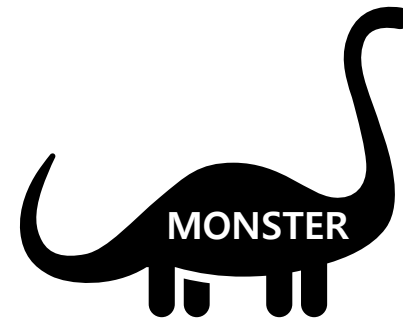


시스템 설계의 시작은 "모듈화"이다.

내가 만들 시스템에서

어떤 모듈들로 구성될지 결정하는 것을 **모듈화**라고 한다.

- 비유 : 내가 만들 영화에서, 어떤 인물이 등장할지 먼저 정하기

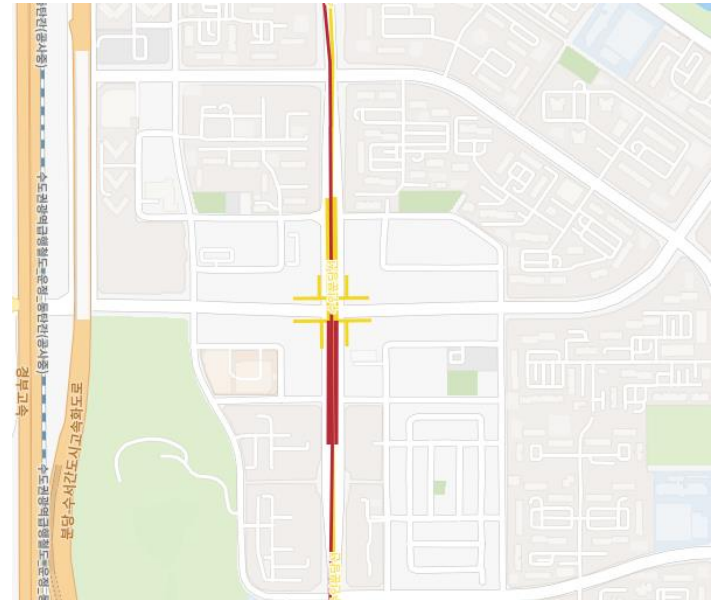
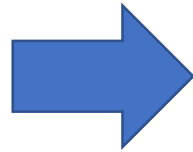


TIP. 모듈화는 추상화가 필수

모듈화는 추상화가 필수이다.

불필요한 정보는 은닉하여, 중요한 정보만 취급한다.

내가 만들 시스템은 어떤 모듈로 구성되어야 할지 고민할 때,
시스템의 등장할 모든 모듈을 고려하지 않는다. 핵심 모듈만 설계한다.



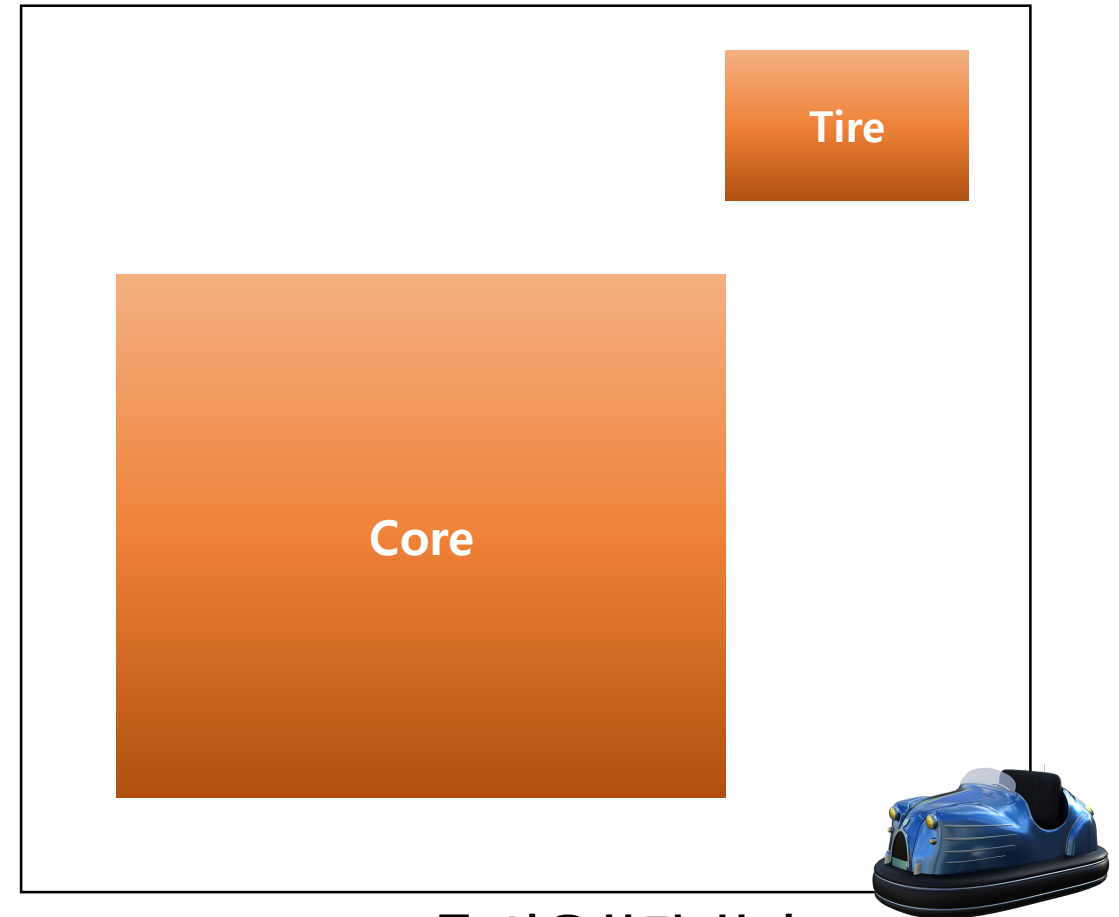
추상화 예시

클린코드! - 리팩토링이 필요한 사례

범퍼카 설계했는데,
다음과 같이 모듈화가 되어 있는 경우
유지보수성이 좋지 않다고 볼 수 있다.

- 버그가 발생한 경우
어떤 모듈에서 버그 발생 확률이 높을까?
- 유지보수 할 경우
어떤 모듈을 수정해야 할 확률이 높을까?

유지보수성을 높이기 위해
명확한 역할을 가진 모듈들로 구성되게끔
리팩토링을 해야한다.

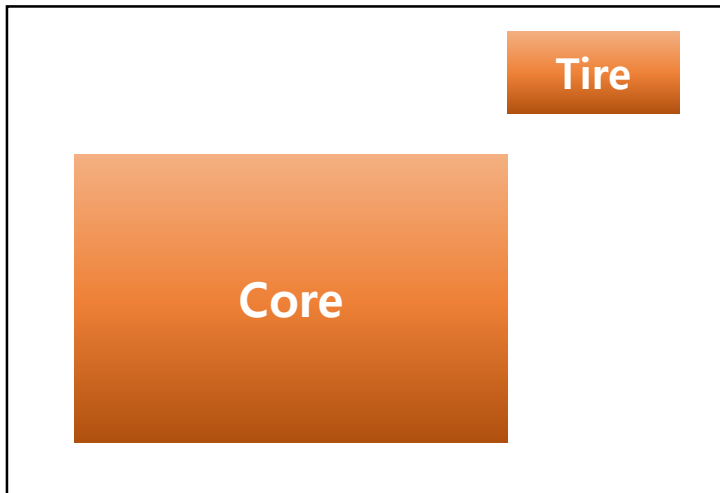


Class를 사용하긴 하나,
절차지향스러운 코드가 되는 모듈 형태

범퍼카 차량 설계하기

명확한 역할을 가진 모듈들로 구성되어있는
시스템으로 모듈화를 해보기

그림판에 간단히 설계해본다.



객체지향스럽지 않은 모듈화



의존도

변경의 의미

소스코드가 **요구사항에 의해** 수정되는 것

1. 메서드 / 필드 수정 / 제거
2. 메서드 / 필드 이름 변경
3. Parameter / Return 값 변경 등등

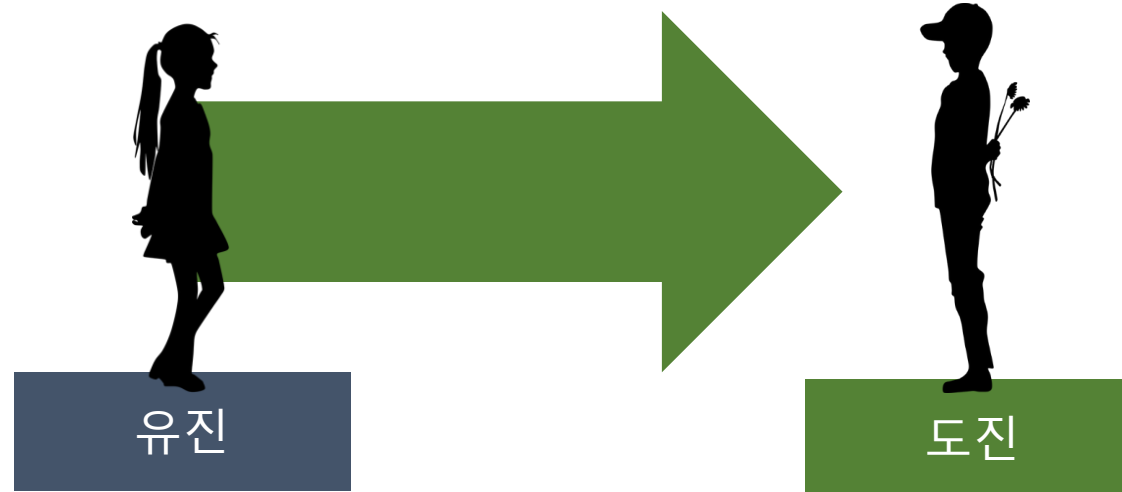
의존도 = 의존성 정도

유진이가 도진에게 의존성을 갖는다.

- 의존성이 크면, 변경에 큰 영향을 미침

ex) 도진이가 힘들면, 유진이 삶에 영향을 끼친다.

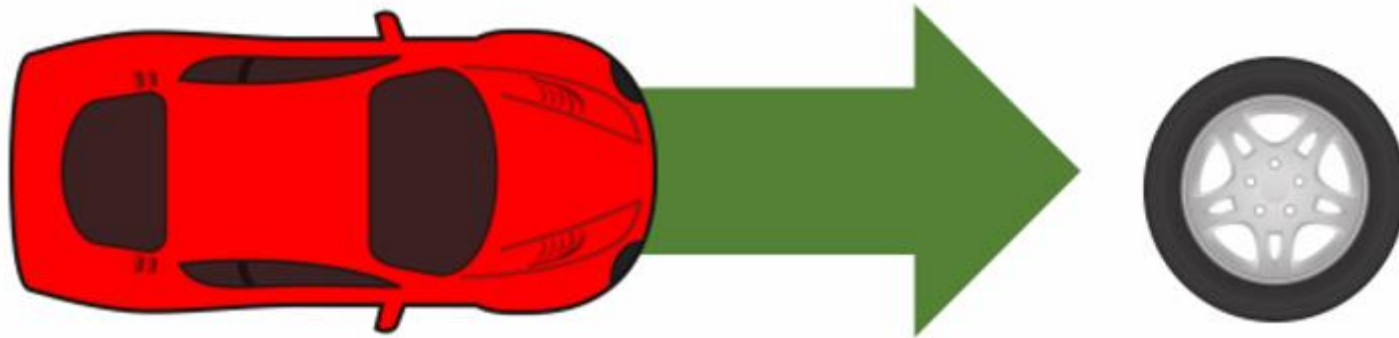
- 의존성이 낮으면, 변경에 큰 영향을 끼치지 않음



다른 클래스에 의존하면?

A가 B Class에 의존할 때, 발생할 수 있는 일

- 만약 B 클래스의 변경이 일어나면, A 클래스도 변경을 해야 할 수 도 있음

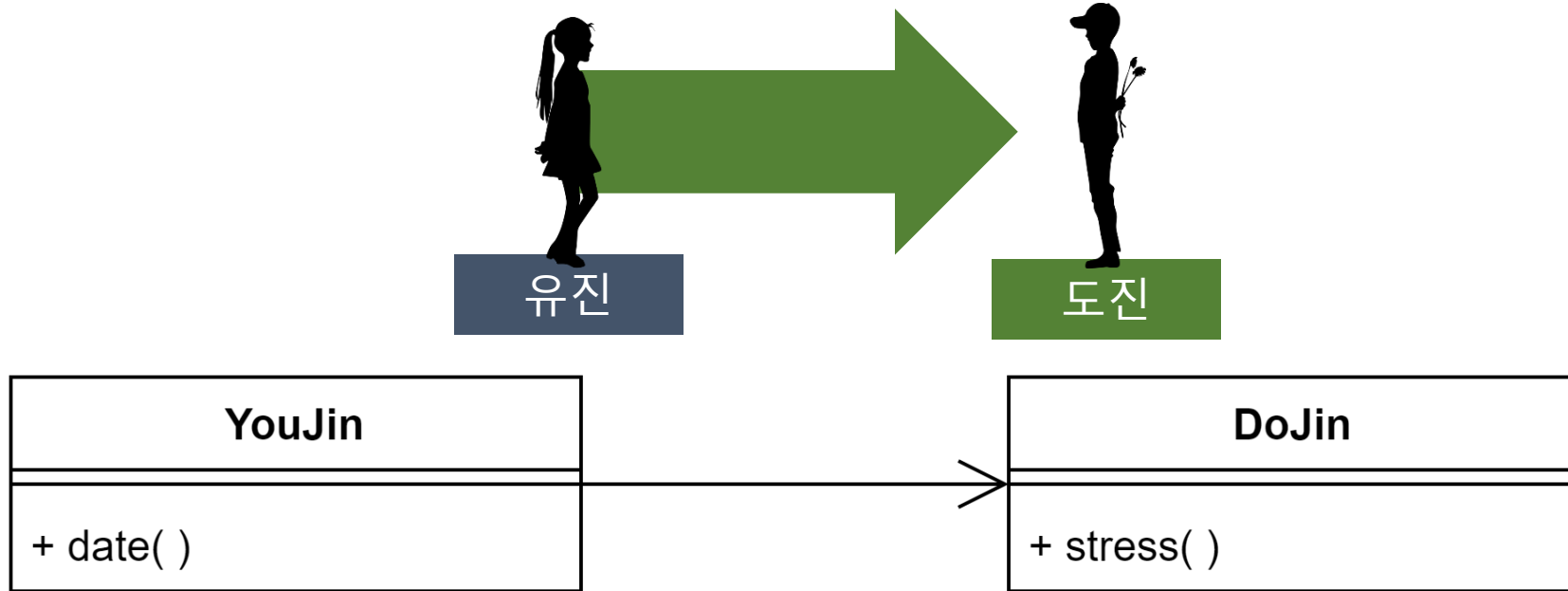


자동차가 바퀴에 의존할 때,
바퀴 사이즈가 변경되면 자동차 설계를 변경해야 할 수도 있음.

의존도

YouJin 이 DoJin 에 대한 의존도가 큰 경우,

DoJin가 변경될 때
YouJin도 함께 변경을 해야 하는 경우가 잦다.



의존도 = 결합도 (Coupling)

의존하는 정도를 결합도 (Coupling) 이라고 부른다.



Loose Coupling
= 낮은 결합도



Tight Coupling
= 높은 결합도

클린코드! - 유지보수성 높이기

만약 A와 B 모듈이 Coupling이 Tight할 경우

B가 변경될 때 A도 변경할 수 있다. (유지보수성이 좋지 않음)

만약 Loose 하도록 리팩토링이 된다면,

B가 변경될 때 A가 변경될 확률이 줄어든다. (유지보수성이 좋아짐)

- 의존도를 낮추는 구현 방법에 대해서는 "리팩토링"시간에 학습한다.



[도전] 객체지향 설계 + 의존관계 방향

냉장고를 추상화 + 모듈화 진행
그리고 각 모듈간 의존관계 방향을 설정



Attribute 와 Behavior

각 모듈은 세부 내용을 갖는다.

Attribute

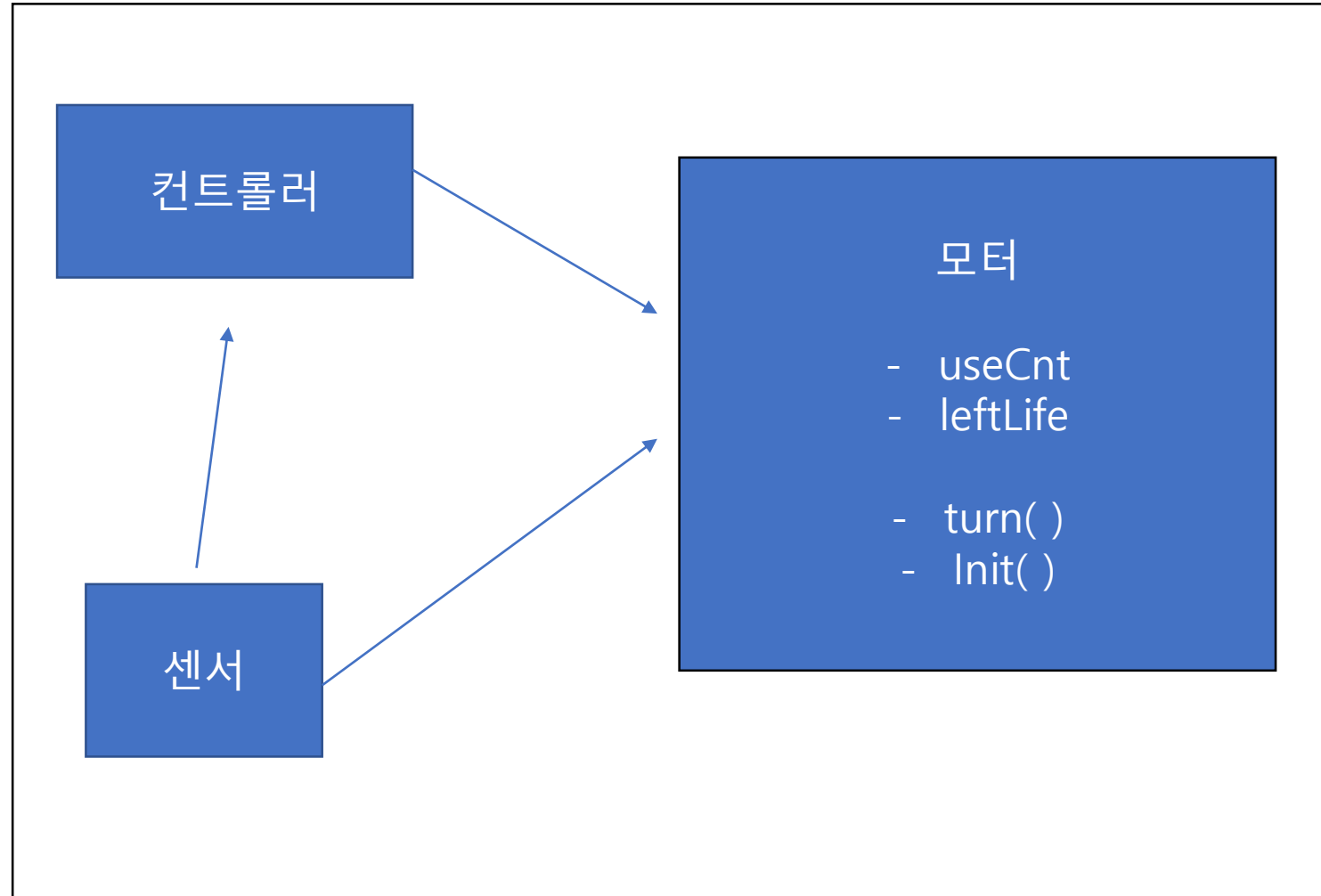
- 데이터 값
- OOP 언어에서 field 로 표현

Behavior

- 동작
- OOP 언어에서
메서드로 표현

추상화 필수!!

- 모든 내용을 넣지 말자



이상한 Attribute

이 모듈에 어울리지 않는 Attribute (??)
이건 설계가 이상하다.

모듈과 관련있는 Attri. / Behavior 만
모여있는 정도를 표현하는 단어는? → Co..

모터

- mousePrice (?)
- lovePersonCount (?)
- turn()
- init()

Cohesion (응집도)

✓모듈이 한 가지 책임을 갖도록 Attri. / Behavior가 구성된 정도

마틴파울러는
모듈이 담당해야하는 임무를
모듈의 “책임(Responsibility)” 이라고 표현했다.

클린코드! - 유지보수성 높이기

모듈이 가진 책임(임무)에 맞지 않는

Attribute / Behavior를 가지고 있는 경우 유지보수성이 좋지 않다.

**Attribute / Behavior 를 적합한 모듈로 옮기는 리팩토링을 통해
유지보수성을 올릴 수 있다.**



다음 멘트 이해하기

✓객체지향 설계는...

- 적절한 추상화가 되어있고,
- 적절한 작은 사이즈의 모듈화가 되어야 하고,
- 각 모듈끼리는 커플링이 루즈할수록 좋으며,
- 하나의 모듈은 Cohesion이 높도록 설계가 이뤄져야 한다.

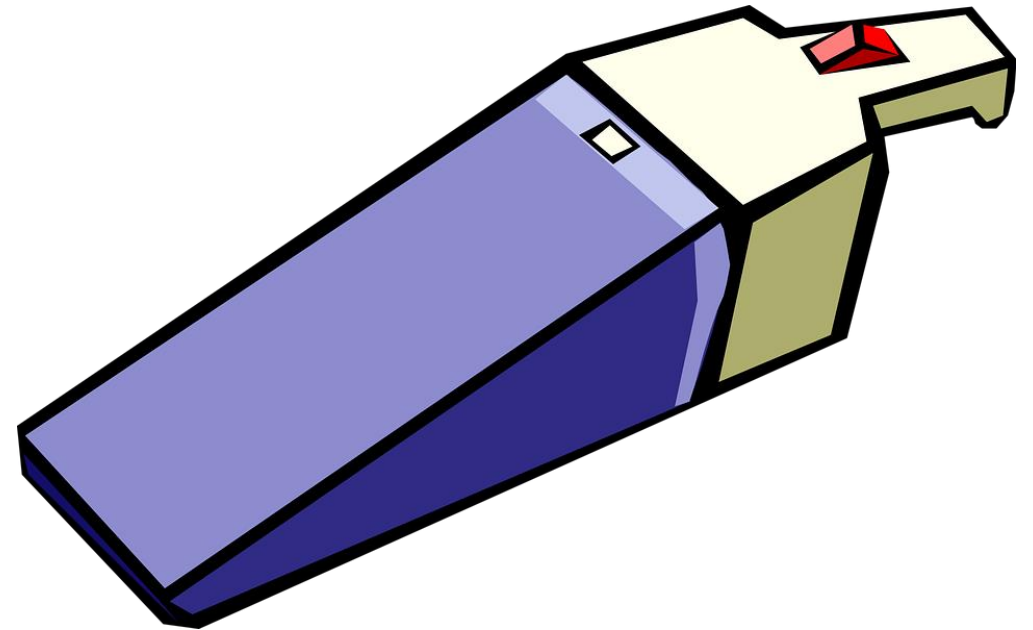
[도전] OOP 설계하기

청소기 설계하기

- 어떤 모듈이 필요하고, 어떤 의존관계 방향인지, 그리고 그 내부 Attribute / Behavior 기입한다.

응집도가 높도록

Attribute / Behavior 기입하기



UML – Class Diagram 예시

분석과 설계에 대한 결과물을 표현하는 공식적인 표현 방법

